

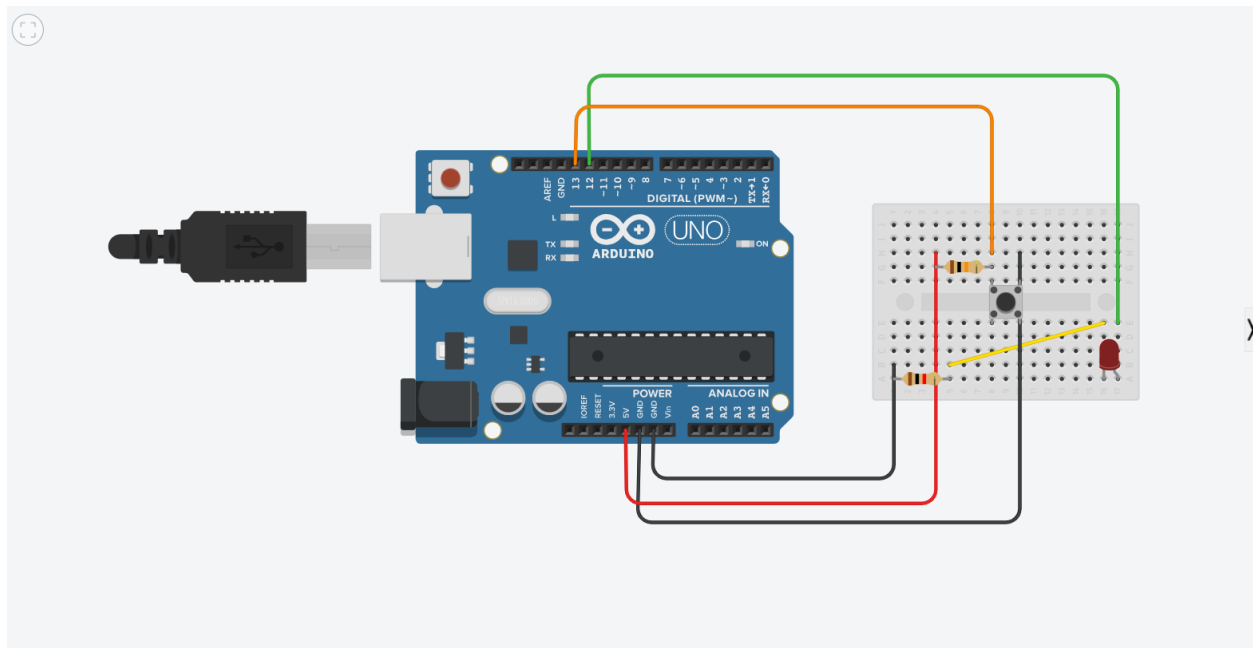
Inginerie Hibridă

Condrea Laurențiu

Tipuriță Alexandru

Palamiuc Ilie

Schemă Arduino



Cod Arduino

```
const int buttonPin = 2; // Button input pin
```

```
const int ledPin = 3;    // LED output pin
```

```
void setup() {
```

```
    pinMode(buttonPin, INPUT_PULLUP); // Use internal pull-up resistor
```

```
    pinMode(ledPin, OUTPUT);          // LED pin as output
```

```
    Serial.begin(115200);             // Serial communication for Simulink
```

```

}

void loop() {
    int buttonState = digitalRead(buttonPin);

    // LED ON when button is pressed
    // With INPUT_PULLUP: pressed = LOW (0), released = HIGH (1)
    if (buttonState == LOW) {
        digitalWrite(ledPin, HIGH);
    } else {
        digitalWrite(ledPin, LOW);
    }

    // Send button state to Simulink
    Serial.write(buttonState);

    delay(5); // Small delay for stable serial output
}

```

Funcție Matlab

```

function [qt,viteza,acceleratie,t] =PickandPlace(Q_initial,Q_final,delta)
%delta=durata totala a traiectoriei
t=linspace(0,delta);
%Coeficienti
a0=Q_initial;
a1=0;
a2=3*(Q_final-Q_initial)/(delta^2);

```

```

a3=-2*(Q_final-Q_initial)/(delta^3);
qt=a3*t.^3+a2*t.^2+a1*t+a0; %pozitie
viteza = 3*a3*t.^2 + 2*a2*t + a1;
acceleratie = 6*a3*t + 2*a2;
figure %pozitie in functie de timp
subplot(3, 1, 1)
plot(t, qt);
xlabel('Time');
ylabel('Pozitie');
grid on
subplot(3, 1, 2) %viteza in functie de timp
plot(t, viteza);
xlabel('Time');
ylabel('Viteza');
grid on
subplot(3, 1, 3) %acceleratie in functie de timp
plot(t, acceleratie);
xlabel('Time');
ylabel('Acceleratie');
grid on
End

```

Apelare funcție:

%Calculul variabilelor generalizare(pick and place)

%intrare:coordonate puncte,durata

%iesire :pozitie,viteza si acceleratie

Q_initial = 0; % Punct de plecare

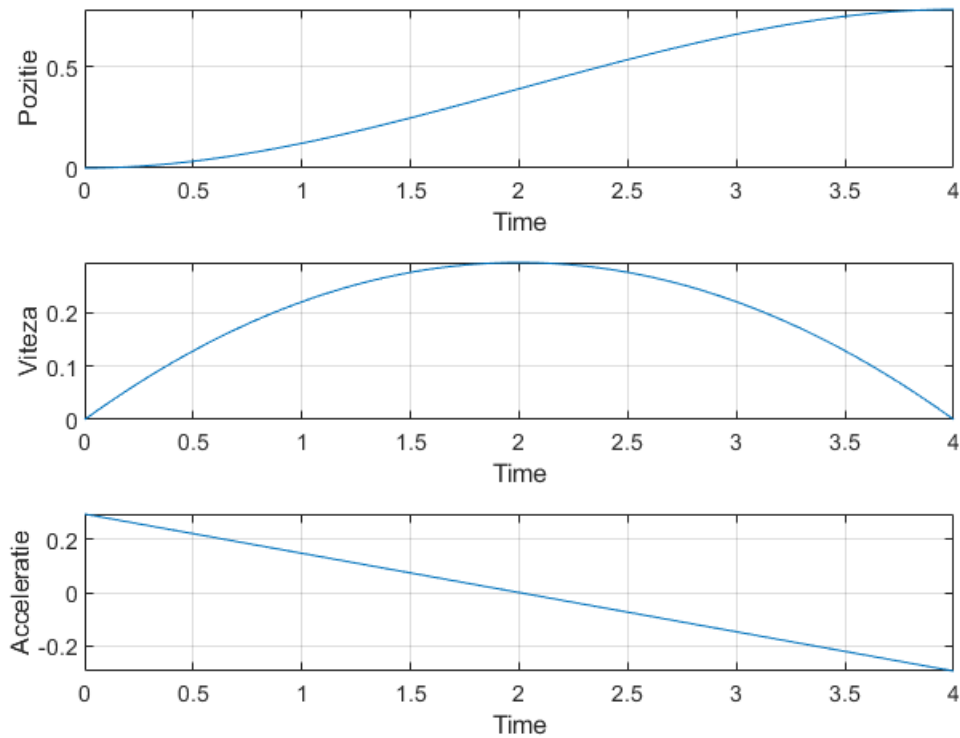
Q_final = pi/4;

[qt,viteza,acceleratie,t]=PickandPlace(Q_initial,Q_final,4)

```
qt1=[t',qt(1,:)']
```

```
v1=[t',viteza(1,:)']
```

```
acc1=[t',acceleratie(1,:)']
```



Comunicarea cu Arduino - Matlab

1. Blocul Serial Configuration

Acesta este blocul de configurare serială pentru Arduino.

Rol:

- deschide comunicația serială pe:
 - port COM5
 - baud rate 115200

Practic:

- Simulink ↔ Arduino

Fără acest bloc, Simulink nu poate comunica cu Arduino.

2. Blocul Serial Receive

Acesta citește datele trimise de Arduino.

Ieșiri:

- Data → valoarea primită
- Status → statusul comunicației

Aici:

- Data este semnalul butonului

Exemplu:

- 1 = apăsat
- 0 = neapăsat

3. NOT

Blocul acela inversează semnalul.

Adică:

- dacă primește 1 → scoate 0
- dacă primește 0 → scoate 1

Arduino-ul trimite logică inversă:

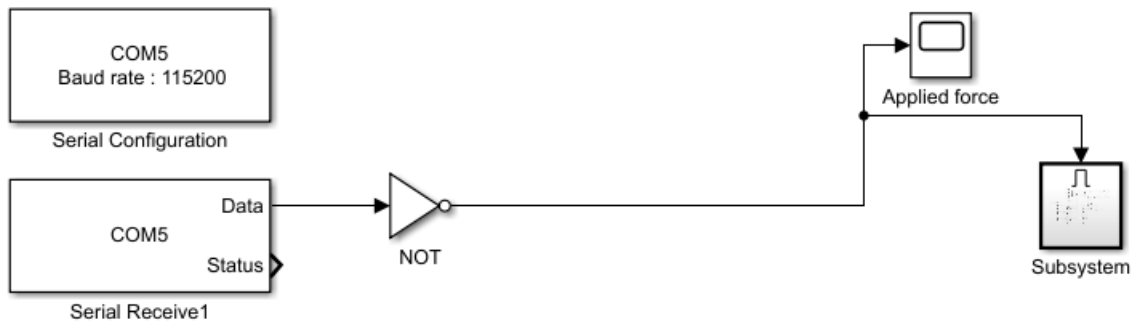
- buton apăsat → 0
- buton liber → 1

și este inversat pentru:

- apăsat → 1
- liber → 0

4. Scope-uri

Afișarea valorilor.



Generare și aplicare a mișcării robotului – Matlab – Subsystem

1. Constant

Bloc constant, scoate permanent valoarea 1.

2. Integrator 1/s

Bloc foarte important, face integrarea în timp.

- transformă valoarea constantă 1 într-un timp care crește continuu

Practic:

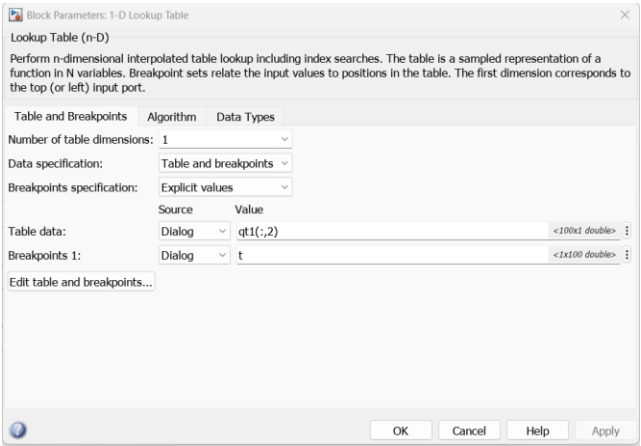
- generează timpul t

Practic, acest bloc funcționează ca un **ceas**.

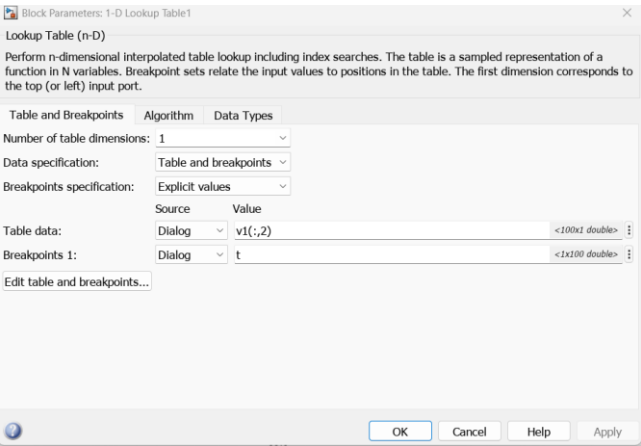
3. 1-D Lookup Table (x6)

Aceste blocuri primesc timpul ca intrare și scot o valoare pe baza unui tabel sau grafic intern prestabilit.

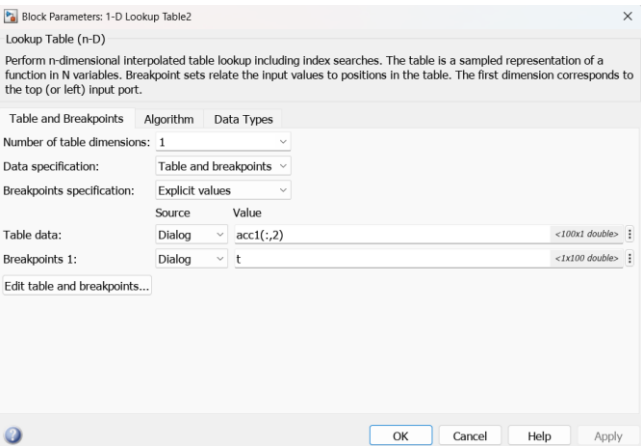
Poziție:



Viteză:



Accelerație:



Block Parameters: 1-D Lookup Table

Lookup Table (n-D)

Perform n-dimensional interpolated table lookup including index searches. The table is a sampled representation of a function in N variables. Breakpoint sets relate the input values to positions in the table. The first dimension corresponds to the top (or left) input port.

Table and Breakpoints Algorithm Data Types

Lookup method

Interpolation method: Linear point-slope ☐ Apply full precision fixed-point algorithm when possible

Extrapolation method: **Clip** ☐ Use last table value for inputs at or above last breakpoint

Index search method: Binary search ☐ Begin index search using previous index result

Diagnostic for out-of-range input: None

Input settings

☐ Use one input port for all inputs (u)

Code generation

☐ Remove protection against out-of-range input in generated code

☐ Support tunable table size in code generation

OK Cancel Help Apply

La toate trebuie selectat Clip la Extrapolation method pentru ca mecanismul **se va opri și își va menține acea ultimă poziție** pentru tot restul simulării sau până când se va apăsa din nou butonul.

4. Gain

Înmulțește semnalul care trece prin el cu -1. Inversează semnalul matematic, ceea ce, în contextul mișcării, se traduce prin schimbarea sensului de rotație pentru acea componentă.

5. Enable

Acest bloc este de obicei folosit în Simulink pentru a transforma un întreg subsistem într-unul condiționat, care se execută doar atunci când i se dă comandă de activare. Se activează opțiunea de reset în interiorul block-ului să înceapă sistemul de la început la fiecare apăsare de buton. Acest block activează output-ul subsystem-ului în care intră semnalul butonului.

6. Simulink-PS Converter

Face puntea între semnalele pur matematice din Simulink și modelul fizic. Ia semnalul de poziție generat de Lookup Tables și îl transformă într-un semnal fizic care comandă portul q al blocurilor Revolute Joint.

7. Solver Configuration

Este creierul simulării fizice. Definește setările matematice necesare pentru a rezolva ecuațiile sistemului mecanic. Orice model Simscape are nevoie de el.

8. Mechanism Configuration

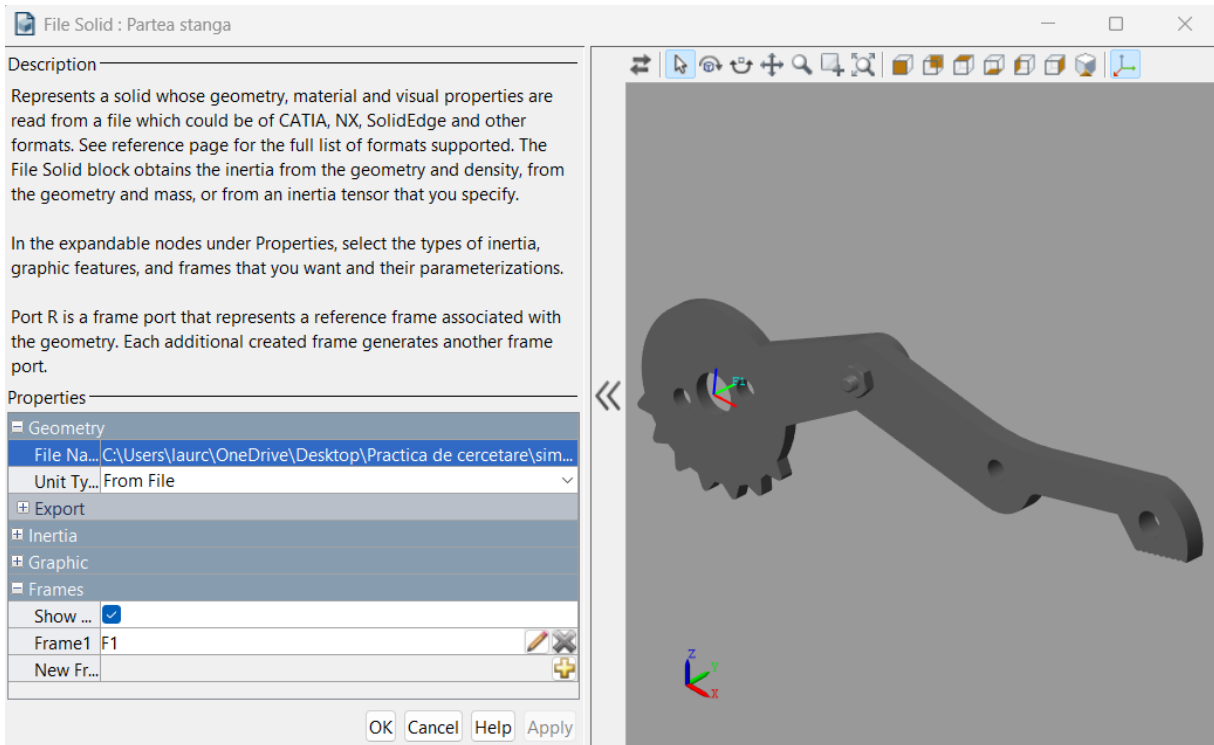
Setează parametrii globali ai mecanismului, cum ar fi direcția și valoarea gravitației mediului în care funcționează sistemul.

9. World Frame:

Reprezintă sistemul de referință global absolut (coordonatele $X=0$, $Y=0$, $Z=0$). Este "pământul" de care se leagă restul mecanismului.

10. File Solid (x3)

Acestea reprezintă corpurile rigide (piesele 3D) ale mecanismului. Ele conțin date despre geometrie, masă, inerție și culori, care sunt importate din fișiere CAD. Poziționăm tot mecanismul cu ajutorul frame-urilor.



11. Rigid Transform (x4):

Aplică o transformare spațială fixă (o translație sau o rotație în spațiul 3D) între două sisteme de coordonate. Aceste blocuri sunt folosite pentru a poziționa exact articulațiile și piesele (File Solid) unele față de celelalte.

12. Revolute Joint (x2):

Reprezintă o articulație de rotație. Permite unui corp să se rotească față de altul pe o singură axă. Observă portul de intrare q – acesta indică faptul că unghiul articulației este controlat din exterior.

Block Parameters: Revolute Joint1

×

Revolute Joint

✓

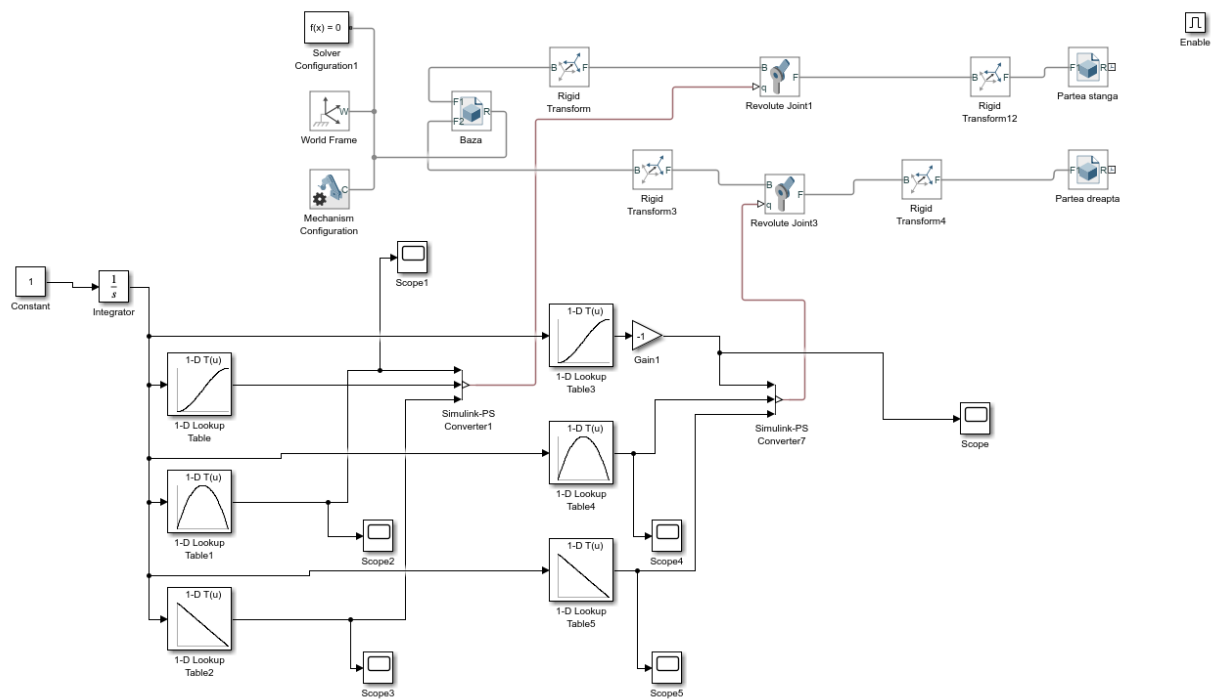
Auto Apply

?

Settings

Description

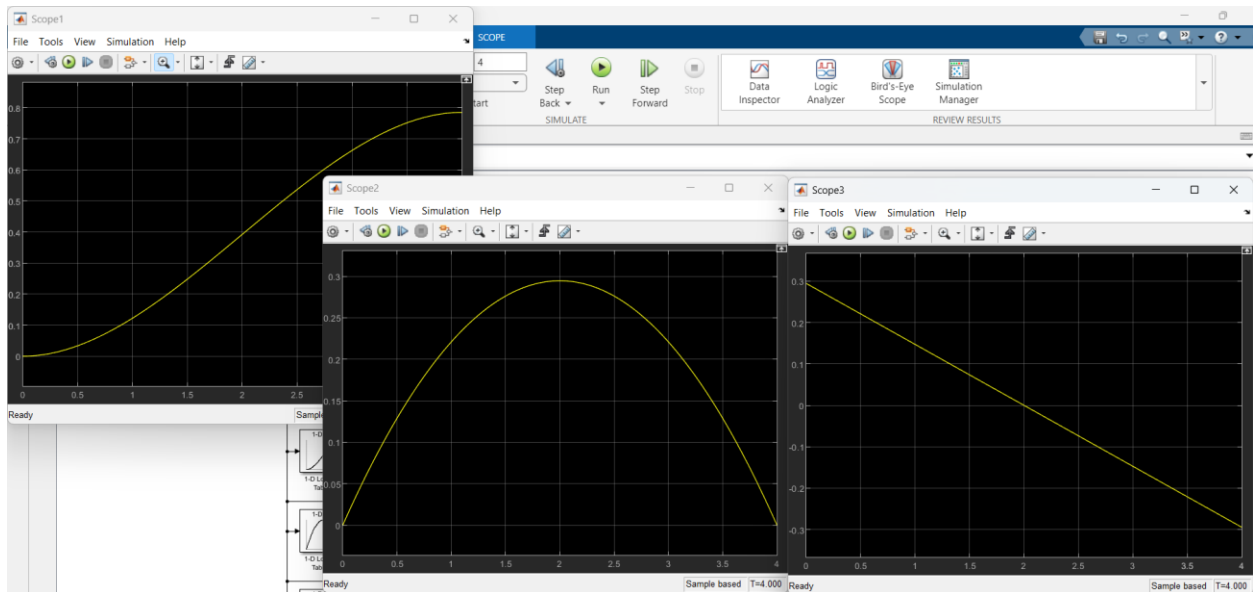
NAME	VALUE
▼ Z Revolute Primitive (Rz)	
> State Targets	
> Internal Mechanics	
> Limits	
▼ Actuation	
Torque	Automatically Computed ▼
Motion	Provided by Input ▼
> Sensing	
> Mode Configuration	
> Composite Force/Torque Sensing	



Rezultate

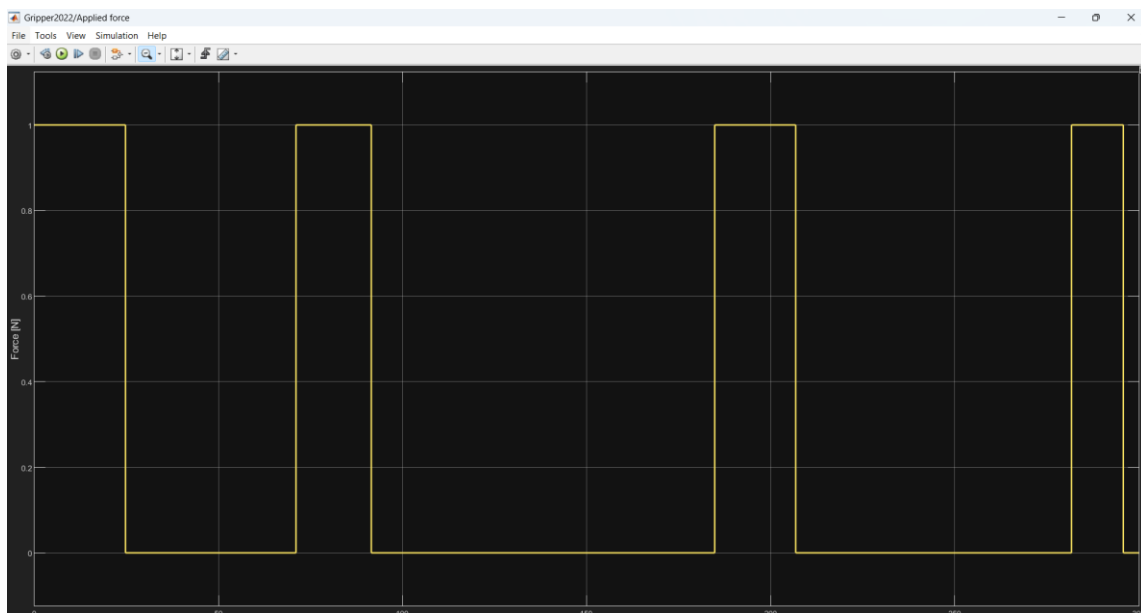
Pentru o perioadă de 4 secunde la simulare se poate observa că obținem aceeași poziție, viteză și accelerație ca în tabele.

Pentru câteva secunde sistemul primește semnal 1 până când acționează poarta NOT, de aceea avem aceste valori fără să fi apăsat pe buton.



Pentru o perioadă de 300s:

-butonul a fost apăsat de 3 ori



Avem 3 semnale de 0 -> poziția, viteză și accelerație dorită.

